Συστήματα Μικροϋπολογιστών

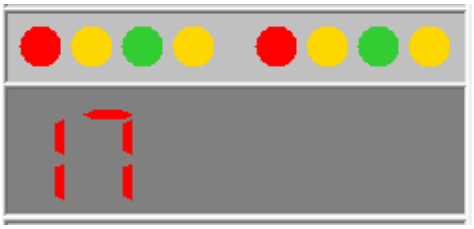
3η Ομάδα Ασκήσεων

Νικόλαος Λοβέρδος (03120442)

Δημήτριος Μιχαλακόπουλος (03120229)

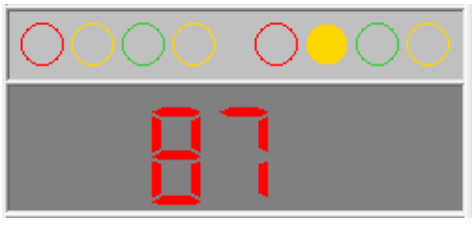
Άσκηση 1

Στο αρχείο **Ex1.8085** του zip βρίσκεται το ζητούμενο πρόγραμμα μαζί με τα απαραίτητα σχόλια. Χρησιμοποιούμε τις εντολές SIM και EI για να επιτραπεί η διακοπή RST 6.5. Όταν πατάμε και αφήνουμε το κουμπί INTRPT προκαλείται διακοπή με αποτέλεσμα τα LEDs να αναβοσβήνουν με περίοδο 0.5 sec και στα δύο αριστερότερα ψηφία των 7-segment displays να αναγράφεται ο υπολειπόμενος χρόνος σε δεκαδική μορφή, το οποίο υλοποιήθηκε με τις ρουτίνες STDM και DCD και με την μέθοδο της επαναλαμβανόμενης αφαίρεσης. Ένα παράδειγμα λειτουργίας του προγράμματος φαίνεται στην εικόνα παρακάτω:



Άσκηση 2

Στο αρχείο **Ex2.8085** του zip βρίσκεται το ζητούμενο πρόγραμμα μαζί με τα απαραίτητα σχόλια. Χρησιμοποιούμε τις ίδιες εντολές για την ενεργοποίηση των διακοπών και την εμφάνιση των αποτελεσμάτων στα 7-segment displays. Αρχικά, ορίζουμε τρεις τυχαίες τιμές στα κατώφλια K1, K2 και K3 στους καταχωρητές C, D και E. Έπειτα το πρόγραμμα περιμένει την διακοπή RST 6.5 για να δεχτεί είσοδο από το πληκτρολόγιο μέσω της ρουτίνας KIND, να την τυπώσει στα δύο μεσαία ψηφία των 7-segment displays και να κάνει την σύγκριση ανάβοντας το αντίστοιχο LED. Ένα παράδειγμα λειτουργίας του προγράμματος φαίνεται στην εικόνα παρακάτω:



Άσκηση 3

α) Η υλοποίηση της ζητούμενης μακροεντολής INR16 ADDR φαίνεται παρακάτω:

```
INR16 MACRO ADDR
    PUSH B
    PUSH H
    LXI H,ADDR  ;load ADDR in HL
    MOV C,M     ;store XLOW in C
    INX H       ;go to next address HL=ADDR+1
    MOV B,M     ;store XHIGH in B
    INX B       ;increase X in BC
    MOV M,B     ;store XHIGH back at ADDR+1
    DCX H       ;go to previous address HL=ADDR
    MOV M,C     ;store XLOW back at ADDR
    POP H
    POP B
ENDM
```

β) Η υλοποίηση της ζητούμενης μακροεντολής FILL ADDR, K φαίνεται παρακάτω:

```
FILL MACRO ADDR,K
    PUSH PSW
    PUSH H
    LXI H,ADDR      ;load ADDR in HL
    MVI A,K         ;load K in Accumulator
    LOOP: MOV M,A   ;store K in address ADDR
    INX H           ;increase address in HL
    DCR A           ;decrease K in Accumulator
    CPI 00H         ;check if A is zero
    JNZ LOOP        ;if not loop
    POP H
    POP PSW
ENDM
```

γ) Η υλοποίηση της ζητούμενης μακροεντολής RHLL Q, R φαίνεται παρακάτω:

```
RHLL MACRO Q,R
    MOV A,R     ;load R in Accumulator
    RAL         ;MSB of R is in CY
    MOV R,A     ;LSB of R is previous CY
    MOV A,Q     ;load Q in Accumulator
    RAL         ;MSB of Q is in CY
    MOV Q,A     ;LSB of Q is previous CY (MSB of R)
ENDM
```

Άσκηση 4

Αρχικά θεωρούμε ότι οι διακοπές είναι ενεργοποιημένες και ότι υπάρχει η κατάλληλη μάσκα. Η RST 6.5 είναι μια διακοπή υλικού (hardware interrupt) που θα λάβει προτεραιότητα έναντι της εκτέλεσης του υπόλοιπου προγράμματος, αρά θα συμβούν κατά σειρά τα παρακάτω:

- Θα ολοκληρωθεί η εκτέλεση της τρέχουσας εντολής άρα θα είναι (PC) = 2200H.
- Ο μετρητής προγράμματος PC θα σωθεί στην στοίβα στις θέσεις μνήμης (SP)-1 και (SP)-2, άρα η τιμή 2200H θα σωθεί ως εξής: (2FFFH) = 22H και (2FFEH) = 00H.
- Το μέγεθος της στοίβας θα αυξηθεί κατά 2, άρα θα είναι (SP) = 2FFEH.
- Η διεύθυνση της διακοπής RST 6.5 θα αποθηκευτεί στον PC, άρα θα είναι (PC) = 0034H.
- Αφού εκτελεστεί η ρουτίνα εξυπηρέτησης της διακοπής, επαναφέρεται η παλιά τιμή του PC από την στοίβα και μειώνεται το μέγεθος της στοίβας κατά 2, άρα αυξάνεται ο SP κατά 2. Έτσι, θα έχουμε τις τιμές (PC) = 2200H και (SP) = 3000H μετά την εκτέλεση της διακοπής και στην συνέχεια γίνεται η εκτέλεση των εντολών στην διεύθυνση της εντολής JMP.

Άσκηση 5

α) Η υλοποίηση του ζητούμενου προγράμματος με διακοπές φαίνεται παρακάτω:

```
RST5.5:
    JMP INTRPT_HANDLER
START:
    MVI D,20H   ;D holds number of inputs (32)
    MVI H,00H   ;H holds MSBs of sum
    MVI L,00H   ;L holds LSBs of sum
    MVI B,00H   ;for DAD
    MVI A,0EH   ;mask for RST 5.5
    SIM
LOOP:
    MOV A,D
    CPI 00H
    JZ RESULT
    EI          ;enable interrupts
WAIT:
    JMP WAIT
INTRPT_HANDLER:
    INX SP
    INX SP      ;reduce stack by 2
    DCR D
    MOV A,D
    ANI 01H
    CPI 00H
```

```
    JZ MSBS     ;check if D is even
    IN 20H      ;if no get LSBS
    ANI F0H     ;keep X7-X4
    RRC
    RRC
    RRC
    RRC         ;shift LSBS to correct position
    MOV C,A     ;and store them in C
    JMP LOOP
MSBS:
    IN 20H      ;if yes get MSBS
    ANI F0H     ;keep X7-X4
    ADD C       ;form the 8-bit number
    MOV C,A     ;store 8-bit number in C
    DAD B       ;add BC to HL (B is 0, C contains an 8-bit number)
    JMP LOOP
RESULT:
    DI
    DAD H       ;add HL to itself 4 times
    DAD H       ;which imitates 4 left shifts
    DAD H       ;H contains the 8-bit result
    DAD H       ;L contains decimal bits
    MVI L,00H   ;truncate decimal part
    RET
END
```

β) Η υλοποίηση του ζητούμενου προγράμματος χωρίς διακοπές φαίνεται παρακάτω:

```
    MVI D,20H   ;D holds number of inputs (32)
    MVI H,00H   ;H holds MSBs of sum
    MVI L,00H   ;L holds LSBs of sum
    MVI B,00H   ;for DAD
START:
    IN 20H
    ANI 01H
    JM START
LSBS:
    MOV A,D
    CPI 00H
    JZ RESULT
    DCR D
    MOV A,D
    ANI 01H
    CPI 00H
    JZ MSBS     ;check if D is even
```

```
    IN 20H      ;if no get LSBS
    ANI F0H     ;keep X7-X4
    RRC
    RRC
    RRC
    RRC         ;shift LSBS to correct position
    MOV C,A     ;and store them in C
    JMP CHECK
MSBS:
    IN 20H      ;if yes get MSBS
    ANI F0H     ;keep X7-X4
    ADD C       ;form the 8-bit number
    MOV C,A     ;store 8-bit number in C
    DAD B       ;add BC to HL (B is 0, C contains an 8-bit number)
CHECK:
    IN 20H
    ANI 01H
    JP CHECK
    JMP START
RESULT:
    DAD H       ;add HL to itself 4 times
    DAD H       ;which imitates 4 left shifts
    DAD H       ;H contains the 8-bit result
    DAD H       ;L contains decimal bits
    MVI L,00H   ;truncate decimal part
END
```